TÀI LIỆU THỰC HÀNH · PYTHON

Thiết lập Linter, Type Checker & CI cho Python với `uv`

Hướng dẫn từng bước cho sinh viên: từ cài `uv` trên máy cá nhân đến khi GitHub Actions tự động kiểm tra mã nguồn và báo ✅ xanh.

BẠN SẼ LÀM ĐƯỢC

- Tạo dự án Python chuẩn bằng `uv`
- Bắt lỗi phong cách & lỗi tiềm ẩn bằng **Ruff**
- Bắt lỗi kiểu dữ liệu bằng **mypy**
- Chặn commit lỗi bằng **pre-commit**
- Chạy tự động trên **GitHub Actions** mỗi lần push

THỜI GIAN

60–90 phút

YÊU CẦU

Biết dùng terminal cơ bản, có tài khoản GitHub

HỆ ĐIỀU HÀNH

Windows, macOS hoặc Linux

Đã chạy thử toàn bộ các bước vào tháng 10/2026 với: Ruff 0.17 · mypy 2.4 · pytest 9.1 · pre-commit 4.7 · `actions/checkout@v7` · `astral-sh/setup-uv@v9`

Mục lục

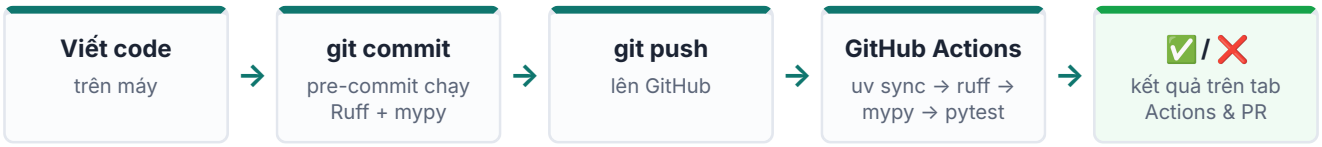
1. Bức tranh tổng thể
2. Chuẩn bị môi trường
3. Cài đặt `uv`
4. Tạo dự án mới
5. Thêm công cụ phát triển
6. Viết mã mẫu (có lỗi cố ý)
7. Linter & Formatter: Ruff
8. Type Checker: mypy
9. Kiểm thử: pytest
10. Chặn lỗi trước khi commit: pre-commit
11. Đưa dự án lên GitHub
12. Viết workflow GitHub Actions
13. Kiểm tra CI hoạt động
14. Xử lý sự cố thường gặp
15. Checklist & bài tập

Phụ lục A – Toàn bộ file cấu hình · Phụ lục B – Bảng lệnh nhanh · Tài liệu tham khảo

1. Bức tranh tổng thể

Trước khi gõ lệnh, hãy hiểu mỗi công cụ làm gì và chúng nối với nhau ra sao.

Công cụ	Vai trò	Ví dụ lỗi nó bắt được
uv	Quản lý Python, môi trường ảo (`.venv`), thư viện và file khóa phiên bản `uv.lock`	— (nền tảng cho mọi thứ)
Ruff	*Linter* (tìm lỗi/mùi code) + *Formatter* (định dạng code thống nhất)	`import` thừa, import sai thứ tự, biến không dùng, cú pháp cũ
mypy	*Type checker* – kiểm tra kiểu dữ liệu mà không cần chạy chương trình	Cộng `int` với `str` , hàm thiếu chú thích kiểu, trả về sai kiểu
pytest	Chạy kiểm thử đơn vị (unit test)	Hàm cho kết quả sai
pre-commit	Tự chạy Ruff + mypy **trước mỗi lần** `git commit` trên máy bạn	Ngăn commit code lỗi
GitHub Actions	Máy chủ của GitHub tự chạy toàn bộ kiểm tra **mỗi lần push / mở Pull Request**	Bắt lỗi khi ai đó quên chạy kiểm tra ở máy



GHI CHÚ

Vì sao cần cả hai lớp (pre-commit và CI)? pre-commit cho phản hồi tức thì trên máy bạn, nhưng có thể bị bỏ qua (`git commit --no-verify`) hoặc chưa được cài ở máy người khác. CI trên GitHub là "trọng tài cuối cùng" – không ai lách được.

2. Chuẩn bị môi trường

Bạn cần ba thứ: **Git**, **tài khoản GitHub** và một **trình soạn thảo** (khuyên dùng VS Code). Kiểm tra Git:

```
git --version
# git version 2.4x.x  ← có dòng này là được
```

Nếu chưa có Git: Windows tải tại *git-scm.com*; macOS chạy `xcode-select --install` ; Ubuntu/Debian chạy `sudo apt install git` .

Khai báo danh tính cho Git (chỉ làm một lần trên mỗi máy):

```
git config --global user.name  "Nguyen Van A"
git config --global user.email "ban@example.com"
git config --global init.defaultBranch main
```

> **MẸO**
> **Tiện ích VS Code nên cài:** *Python* (Microsoft), *Ruff* (Astral) và *Mypy Type Checker* (Microsoft). Chúng gạch chân lỗi ngay khi bạn gõ, dùng chung cấu hình bạn sẽ viết ở các bước sau.

3. Cài đặt `uv`

`uv` là công cụ do Astral (nhóm làm ra Ruff) phát triển, thay thế cho `pip`, `venv`, `pyenv`... trong một chương trình duy nhất, viết bằng Rust nên rất nhanh. **Bạn không cần cài Python trước** – `uv` sẽ tự tải Python khi cần.

3.1. Cài theo hệ điều hành

macOS / Linux – mở Terminal:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows – mở PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Cách khác: `brew install uv` (macOS), `winget install --id=astral-sh.uv -e` (Windows), hoặc `pipx install uv`.

3.2. Kiểm tra

Đóng terminal và mở lại (để biến PATH được cập nhật), rồi chạy:

```
uv --version
# uv 0.x.y (...)
```

> **LƯU Ý**
> **Báo `command not found` / `'uv' is not recognized`?** Hầu như luôn do chưa mở lại terminal. Nếu vẫn lỗi, thêm thư mục `~/.local/bin` (macOS/Linux) hoặc `%USERPROFILE%\.local\bin` (Windows) vào biến môi trường PATH.

Cập nhật `uv` về sau (nếu cài bằng script chính thức): `uv self update`.

4. Tạo dự án mới

4.1. Khởi tạo

```
uv init --lib --python 3.13 demo-ci
cd demo-ci
```

- `--lib` tạo cấu trúc **src layout** (code nằm trong `src/`), là cấu trúc được khuyến nghị vì giúp test luôn chạy trên gói đã cài, tránh lỗi import "chạy được ở máy tôi".
- `--python 3.13` ghim phiên bản Python. Nếu máy chưa có Python 3.13, `uv` tự tải về.
- `uv init` cũng tự chạy `git init` cho bạn.

4.2. Cấu trúc được tạo ra

```
demo-ci/
├── .git/               ← kho Git (uv init tạo sẵn)
├── .gitignore          ← đã bỏ qua .venv, __pycache__, ...
├── .python-version     ← "3.13": phiên bản Python của dự án
├── README.md
├── pyproject.toml      ← "trái tim" dự án: tên, phụ thuộc, cấu hình công cụ
└── src/
    └── demo_ci/
        ├── __init__.py
        └── py.typed    ← đánh dấu gói có chú thích kiểu (cho mypy)
```

Mở `pyproject.toml`, bạn sẽ thấy nội dung tương tự:

```
[project]
name = "demo-ci"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []

[build-system]
requires = ["uv_build>=0.x,<0.y"]   # số phiên bản tùy bản uv của bạn
build-backend = "uv_build"
```

5. Thêm công cụ phát triển

Ruff, mypy, pytest, pre-commit chỉ cần khi **phát triển**, không cần khi người khác *dùng* thư viện của bạn. Vì vậy ta thêm chúng vào nhóm `dev`:

```
uv add --dev ruff mypy pytest pre-commit
```

Lệnh này làm ba việc cùng lúc:

1. Tạo môi trường ảo `.venv/` (nếu chưa có) và cài các gói vào đó.

2. Ghi vào `pyproject.toml` một mục mới `[dependency-groups]` (xem bên dưới).
3. Tạo file **`uv.lock`** – ghi lại *chính xác* phiên bản mọi gói (kể cả gói phụ thuộc gián tiếp).

```
[dependency-groups]
dev = [
    "mypy>=2.4.0",
    "pre-commit>=4.7.0",
    "pytest>=9.1.1",
    "ruff>=0.17.0",
]
```

QUAN TRỌNG

Luôn commit `uv.lock` lên Git. Nhờ file này, máy bạn, máy bạn cùng nhóm và máy chủ GitHub Actions cài **cùng y hệt** phiên bản Ruff/mypy → kết quả kiểm tra giống nhau ở mọi nơi.

Chạy công cụ bằng `uv run` – không cần "activate" môi trường ảo:

```
uv run ruff --version     # ruff 0.17.0
uv run mypy --version     # mypy 2.4.0 (compiled: yes)
uv run pytest --version   # pytest 9.1.1
```

MẸO

`uv run <lệnh>` luôn tự đồng bộ `.venv` với `uv.lock` trước khi chạy. Khi bạn cùng nhóm vừa clone dự án về, chỉ cần `uv sync` là có đủ môi trường.

6. Viết mã mẫu (có lỗi cố ý)

Để thấy công cụ hoạt động, ta viết một file **cố tình có lỗi**. Tạo `src/demo_ci/calculator.py`:

```
import os, sys
from typing import List


def average(numbers: List[float]) -> float:
    if len(numbers) == 0:
        return 0
    return sum(numbers) / len(numbers)


def greet(name):
    return "Xin chào, " + name


def total_length(words: list[str]) -> int:
    return sum(len(w) for w in words) + "0"
```

Tạo thư mục `tests/` và file `tests/test_calculator.py`:

```python
from demo_ci.calculator import average, greet


def test_average() -> None:
    assert average([1.0, 2.0, 3.0]) == 2.0


def test_average_empty() -> None:
    assert average([]) == 0


def test_greet() -> None:
    assert greet("An") == "Xin chào, An"
```

Đoạn code trên có những vấn đề sau – hãy thử tự tìm trước khi để công cụ chỉ ra:

Vị trí	Vấn đề	Công cụ phát hiện
Dòng 1	Import 2 module trên một dòng, và cả hai không được dùng	Ruff
Dòng 2	`typing.List` đã lỗi thời, nên dùng `list`	Ruff
`greet`	Thiếu chú thích kiểu cho tham số và giá trị trả về	mypy
`total_length`	Cộng `int` với `str` → sẽ gây `TypeError` khi chạy	mypy

7. Linter & Formatter: Ruff

7.1. Cấu hình

Thêm vào **cuối** `pyproject.toml`:

```toml
# ---------------- Ruff: linter + formatter ----------------
[tool.ruff]
line-length = 88
target-version = "py313"
src = ["src", "tests"]

[tool.ruff.lint]
select = [
    "E", "W",  # pycodestyle: lỗi phong cách PEP 8
    "F",       # Pyflakes: import thừa, biến không dùng, tên chưa định nghĩa
    "I",       # isort: sắp xếp import
    "B",       # flake8-bugbear: các "bẫy" hay gặp
    "UP",      # pyupgrade: dùng cú pháp Python hiện đại
    "SIM",     # flake8-simplify: gợi ý viết gọn hơn
    "N",       # pep8-naming: quy tắc đặt tên
]

[tool.ruff.format]
quote-style = "double"
```

> **GHI CHÚ**
> Mặc định Ruff chỉ bật nhóm `E` (một phần) và `F`. Danh sách `select` ở trên là bộ cân bằng cho sinh viên: đủ nghiêm để học thói quen tốt, chưa đến mức "phiền". Tra cứu ý nghĩa từng mã lỗi bằng lệnh `uv run ruff rule F401`.

7.2. Chạy linter

```
uv run ruff check --output-format concise
```

Kết quả (đã chạy thật):

```
src/demo_ci/calculator.py:1:1: E401 [*] Multiple imports on one line
src/demo_ci/calculator.py:1:1: I001 [*] Import block is un-sorted or un-formatted
src/demo_ci/calculator.py:1:8: F401 [*] `os` imported but unused
src/demo_ci/calculator.py:1:12: F401 [*] `sys` imported but unused
src/demo_ci/calculator.py:2:1: UP035 `typing.List` is deprecated, use `list` instead
src/demo_ci/calculator.py:5:22: UP006 [*] Use `list` instead of `List` for type annotation
Found 6 errors.
[*] 5 fixable with `--fix`.
```

Cách đọc: `file:dòng:cột: MÃ_LỖI mô tả`. Dấu `[*]` nghĩa là Ruff tự sửa được. Bỏ `--output-format concise` để xem giải thích chi tiết kèm gợi ý sửa.

7.3. Tự động sửa và định dạng

```
uv run ruff check --fix    # Found 5 errors (5 fixed, 0 remaining).
uv run ruff format         # định dạng lại toàn bộ code
```

Lỗi `UP035` còn lại sẽ tự hết vì `--fix` đã xóa dòng `from typing import List` khi không còn dùng đến. Chạy lại `uv run ruff check` → `All checks passed!`

> **MẸO**
> **`ruff check` vs `ruff format`:** `check` tìm *lỗi logic/phong cách* (import thừa, tên biến...). `format` chỉ lo *trình bày* (dấu nháy, xuống dòng, khoảng trắng) – giống Black. Trong CI ta dùng `ruff format --check` để *kiểm tra* mà không sửa file.

8. Type Checker: mypy

8.1. Cấu hình

Thêm tiếp vào `pyproject.toml`:

```
# ---------------- mypy: type checker ----------------
[tool.mypy]
python_version = "3.13"
strict = true
files = ["src", "tests"]
```

`strict = true` bật toàn bộ kiểm tra nghiêm ngặt, trong đó quan trọng nhất là **bắt buộc mọi hàm phải có chú thích kiểu**. Với dự án mới, hãy bắt đầu với `strict` – thêm sau sẽ khó hơn nhiều.

8.2. Chạy mypy

```
uv run mypy
```

```
src/demo_ci/calculator.py:11: error: Function is missing a type annotation  [no-untyped-def]
src/demo_ci/calculator.py:16: error: Unsupported operand types for + ("int" and "str")
[operator]
tests/test_calculator.py:13: error: Call to untyped function "greet" in typed context  [no-
untyped-call]
Found 3 errors in 2 files (checked 3 source files)
```

Lỗi thứ hai là **bug thật**: chương trình sẽ hỏng khi gọi `total_length`, dù chưa có test nào chạm tới nó. Đây chính là giá trị của type checker.

8.3. Sửa code

Nội dung `src/demo_ci/calculator.py` sau khi sửa:

```python
def average(numbers: list[float]) -> float:
    if len(numbers) == 0:
        return 0
    return sum(numbers) / len(numbers)


def greet(name: str) -> str:
    return "Xin chào, " + name


def total_length(words: list[str]) -> int:
    return sum(len(w) for w in words)
```

Chạy lại: `uv run mypy` → `Success: no issues found in 3 source files`

> **GHI CHÚ**
>
> **Các lựa chọn khác ngoài mypy.** *Pyright* (của Microsoft, là lõi của Pylance trong VS Code) và *ty* (của Astral, rất nhanh). Tính đến tháng 10/2026, ty vẫn ở giai đoạn beta với các bản 0.0.x, nên tài liệu này dùng mypy – công cụ lâu đời và ổn định nhất. Khi đã quen, bạn có thể thử `uvx ty check` để so sánh.

9. Kiểm thử: pytest

Thêm cấu hình pytest vào `pyproject.toml`:

```toml
# ---------------- pytest ----------------
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

Chạy:

```
uv run pytest
```

```
...                                                                  [100%]
3 passed in 0.01s
```

Mỗi dấu `.` là một test đạt; test lỗi hiện chữ `F` kèm chi tiết. Đến đây, cả bốn lệnh kiểm tra đều "xanh" ở máy bạn:

```
uv run ruff check && uv run ruff format --check && uv run mypy && uv run pytest
```

10. Chặn lỗi trước khi commit: pre-commit

10.1. Tạo file cấu hình

Tạo file `.pre-commit-config.yaml` ở thư mục gốc dự án:

```yaml
# Dùng hook "local" để chạy đúng phiên bản ruff/mypy đã khóa trong uv.lock
repos:
  - repo: local
    hooks:
      - id: ruff-check
        name: ruff check
        entry: uv run ruff check --fix
        language: system
        types: [python]
      - id: ruff-format
        name: ruff format
        entry: uv run ruff format
        language: system
        types: [python]
      - id: mypy
        name: mypy
        entry: uv run mypy
        language: system
        types: [python]
        pass_filenames: false
```

GHI CHÚ

Vì sao dùng `repo: local` + `uv run` ? Cách phổ biến khác là trỏ tới repo `astral-sh/ruff-pre-commit`, nhưng khi đó pre-commit tự cài *một bản Ruff riêng* có thể khác phiên bản trong `uv.lock` → máy bạn "xanh" nhưng CI "đỏ". Dùng `uv run` đảm bảo **một nguồn phiên bản duy nhất**. `pass_filenames: false` để mypy kiểm tra cả dự án theo mục `files` trong `pyproject.toml`, tránh bỏ sót lỗi liên file.

10.2. Cài hook vào Git và chạy thử

```
uv run pre-commit install
# pre-commit installed at .git/hooks/pre-commit

uv run pre-commit run --all-files
```

```
ruff check..............................................................Passed
ruff format.............................................................Passed
mypy....................................................................Passed
```

10.3. Thấy pre-commit chặn commit lỗi

Thử tạo file lỗi `src/demo_ci/bad.py`:

```python
import os


def add(a: int, b: int) -> int:
    return a + str(b)
```

```
git add -A
git commit -m "thử lỗi"
```

```
ruff check..............................................................Failed
- hook id: ruff-check
- files were modified by this hook
Found 1 error (1 fixed, 0 remaining).

ruff format.............................................................Failed
- hook id: ruff-format
- files were modified by this hook
1 file reformatted

mypy....................................................................Failed
- hook id: mypy
- exit code: 1
src/demo_ci/bad.py:2: error: Unsupported operand types for + ("int" and "str")  [operator]
```

Commit **bị từ chối**. Ruff đã tự sửa phần nó sửa được (xóa `import os`), còn lỗi kiểu thì bạn phải tự sửa. Quy trình chuẩn khi bị chặn: *sửa lỗi* → `git add` *lại* → `git commit` *lại*. Xong thử nghiệm, xóa file `bad.py`.

11. Đưa dự án lên GitHub

11.1. Commit lần đầu

Kiểm tra trước những gì sẽ được commit – phải có `uv.lock`, **không** có `.venv/`:

```
git add -A
git status
git commit -m "Khởi tạo dự án với uv, ruff, mypy, pytest, pre-commit"
git branch -M main
```

11.2. Tạo repository trên GitHub

Cách 1 – Giao diện web: vào *github.com* → nút **New** (hoặc **+** → **New repository**) → đặt tên `demo-ci` → **không** tích "Add a README / .gitignore / license" (vì dự án đã có sẵn) → **Create repository**. Sau đó chạy (thay `<user>` bằng tên tài khoản của bạn):

```
git remote add origin https://github.com/<user>/demo-ci.git
git push -u origin main
```

Cách 2 – GitHub CLI (nếu đã cài `gh` và chạy `gh auth login`):

```
gh repo create demo-ci --public --source=. --remote=origin --push
```

> **LƯU Ý**
> **Push bị hỏi mật khẩu rồi báo lỗi?** GitHub không nhận mật khẩu tài khoản qua HTTPS. Hãy dùng *Personal Access Token* thay cho mật khẩu, đăng nhập bằng `gh auth login`, hoặc cấu hình SSH key.

12. Viết workflow GitHub Actions

GitHub Actions đọc mọi file `.yml` trong thư mục `.github/workflows/`. Tạo file `.github/workflows/ci.yml`:

```yaml
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:        # cho phép bấm chạy tay trên tab Actions

jobs:
  quality:
    name: Lint, type-check & test (Python ${{ matrix.python-version }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.13", "3.14"]

    steps:
      - name: Checkout code
        uses: actions/checkout@v7

      - name: Install uv
        uses: astral-sh/setup-uv@v9
        with:
          enable-cache: true
          python-version: ${{ matrix.python-version }}

      - name: Install dependencies
        run: uv sync --locked --all-extras --dev

      - name: Ruff - lint
        run: uv run ruff check --output-format=github

      - name: Ruff - format check
        run: uv run ruff format --check --diff

      - name: mypy - type check
```

```
        run: uv run mypy

      - name: pytest - unit tests
        run: uv run pytest
```

12.1. Giải thích từng phần

Dòng / khối	Ý nghĩa
`on: push / pull_request`	Chạy khi push lên `main` hoặc khi có Pull Request vào `main`
`workflow_dispatch`	Thêm nút **Run workflow** để chạy thủ công
`runs-on: ubuntu-latest`	Chạy trên máy ảo Ubuntu miễn phí của GitHub
`matrix.python-version`	Chạy song song 2 job, mỗi job một phiên bản Python
`fail-fast: false`	Một phiên bản lỗi thì phiên bản kia vẫn chạy tiếp để bạn thấy đủ kết quả
`actions/checkout@v7`	Tải mã nguồn của repo về máy ảo
`astral-sh/setup-uv@v9`	Action chính thức cài `uv`; `python-version` đặt Python cho job, `enable-cache` lưu cache để lần sau nhanh hơn
`uv sync --locked`	Cài đúng theo `uv.lock`; **báo lỗi nếu `uv.lock` không khớp `pyproject.toml`** (quên commit lock)
`--output-format=github`	Ruff in lỗi theo định dạng mà GitHub hiển thị thành chú thích ngay trên dòng code
`ruff format --check --diff`	Chỉ kiểm tra, không sửa; in ra phần khác biệt nếu code chưa được format

> **MẸO**
>
> **Ghim phiên bản an toàn hơn (nâng cao).** Tài liệu chính thức của uv ghim action bằng mã commit SHA, ví dụ `actions/checkout@<sha> # v7.0.1`, để tránh việc tag bị thay đổi. Với bài tập, dùng tag `@v7`, `@v9` là đủ và dễ đọc.

12.2. Kiểm tra file YAML trước khi push (tùy chọn)

Lỗi thụt lề YAML là nguyên nhân hàng đầu khiến workflow không chạy. Bạn có thể kiểm tra nhanh bằng `actionlint`:

```
uvx --from actionlint-py actionlint
# không in gì = không có lỗi
```

12.3. Commit và push

```
git add .github/workflows/ci.yml
git commit -m "Thêm CI với GitHub Actions"
git push
```

13. Kiểm tra CI hoạt động

13.1. Xem kết quả lần chạy

1. Mở repo trên GitHub → tab **Actions**.
2. Bạn sẽ thấy lần chạy tên **CI** ứng với commit vừa push, kèm biểu tượng: 🟡 đang chạy, ✅ thành công, ❌ thất bại.
3. Bấm vào lần chạy → thấy 2 job *(Python 3.13)* và *(Python 3.14)*.
4. Bấm vào một job → mở từng bước (*Install uv*, *Ruff - lint*...) để đọc log.

Lần chạy đầu thường mất khoảng 30–60 giây; các lần sau nhanh hơn nhờ cache. Trên trang chính của repo, cạnh commit mới nhất cũng có dấu ✅.

13.2. Thử làm CI thất bại (bắt buộc làm!)

CI chỉ có giá trị khi nó **thực sự bắt được lỗi**. Hãy chứng minh điều đó qua một Pull Request:

```
git switch -c thu-loi-ci
```

Thêm vào cuối `calculator.py` một hàm lỗi kiểu:

```
def double(x: int) -> int:
    return str(x * 2)
```

Commit, **bỏ qua pre-commit** (chỉ để giả lập người quên chạy kiểm tra) và push:

```
git commit -am "Cố ý gây lỗi kiểu" --no-verify
git push -u origin thu-loi-ci
```

Trên GitHub, bấm **Compare & pull request** → **Create pull request**. Sau khoảng 1 phút:

- Phần cuối trang PR hiện **"Some checks were not successful"** với dấu ❌.
- Bấm **Details** → bước *mypy - type check* màu đỏ, log có dòng `error: Incompatible return value type (got "str", expected "int")`.
- Với lỗi Ruff, tab **Files changed** của PR còn hiện chú thích ngay trên dòng code bị lỗi (nhờ `--output-format=github`).

Sửa lại hàm (`return x * 2`), commit và push lên cùng nhánh → CI tự chạy lại và chuyển ✅. Sau đó bạn có thể merge PR hoặc đóng nó và xóa nhánh.

13.3. Gắn huy hiệu (badge) trạng thái vào README

Thêm dòng sau vào đầu `README.md` (thay `<user>`):

```
![CI](https://github.com/<user>/demo-ci/actions/workflows/ci.yml/badge.svg)
```

Huy hiệu tự hiển thị *passing* (xanh) hoặc *failing* (đỏ) theo lần chạy mới nhất trên nhánh mặc định.

13.4. Bắt buộc CI xanh mới được merge (khuyến nghị cho làm nhóm)

Vào **Settings** của repo → mục quy tắc nhánh (*Rules* → *Rulesets*, hoặc *Branches* tùy giao diện) → tạo quy tắc cho nhánh `main` → bật **Require status checks to pass** và chọn hai check *Lint, type-check & test (Python 3.13 / 3.14)*. Từ giờ nút **Merge** bị khóa cho tới khi CI xanh.

> **GHI CHÚ**
> Tên các check chỉ xuất hiện trong danh sách chọn **sau khi** workflow đã chạy ít nhất một lần.

14. Xử lý sự cố thường gặp

Triệu chứng	Nguyên nhân & cách sửa
Tab Actions trống, workflow không chạy	File không nằm đúng `.github/workflows/` (chú ý dấu chấm đầu `.github`), hoặc đuôi file không phải `.yml` / `.yaml`, hoặc nhánh của bạn tên `master` trong khi workflow lọc `main`.
`Invalid workflow file`	Sai thụt lề YAML (phải dùng dấu cách, không dùng Tab). Chạy `actionlint` để biết dòng lỗi.
CI lỗi ở `uv sync --locked`: *The lockfile needs to be updated*	Bạn sửa `pyproject.toml` mà chưa cập nhật lock. Chạy `uv lock` ở máy, commit `uv.lock`, push lại.
Máy xanh nhưng CI đỏ ở Ruff/mypy	Phiên bản công cụ khác nhau (chưa commit `uv.lock`), hoặc pre-commit dùng bản Ruff riêng. Dùng cấu hình `repo: local` ở mục 10.
CI đỏ ở `ruff format --check`	Bạn chưa format. Chạy `uv run ruff format` rồi commit.
mypy: *Cannot find implementation or library stub for module "requests"*	Thư viện thiếu thông tin kiểu. Cài gói stub: `uv add --dev types-requests`. Nếu không có stub, thêm vào `pyproject.toml`: `[[tool.mypy.overrides]]` với `module = ["ten_thu_vien.*"]` và `ignore_missing_imports = true`.
pre-commit báo `uv: command not found` khi commit từ VS Code/ GitHub Desktop	Ứng dụng giao diện không thấy PATH của terminal. Mở lại ứng dụng từ terminal, hoặc thêm thư mục chứa `uv` vào PATH hệ thống.
Windows: PowerShell báo *running scripts is disabled* khi cài `uv`	Dùng đúng lệnh ở mục 3.1 (có `-ExecutionPolicy ByPass`), hoặc cài bằng `winget install --id=astral-sh.uv -e`.
Test báo `ModuleNotFoundError: No module named 'demo_ci'`	Bạn chạy `pytest` trực tiếp thay vì `uv run pytest`, nên không dùng môi trường `.venv` có gói đã cài.
CI bị hủy ở job Python 3.14 do một thư viện chưa hỗ trợ	Tạm thời bỏ `"3.14"` khỏi `matrix` hoặc chờ thư viện cập nhật.

15. Checklist & bài tập

15.1. Checklist hoàn thành

☐ `uv --version` chạy được

☐ Dự án có `pyproject.toml`, `uv.lock`, `.python-version`, thư mục `src/` và `tests/`

☐ `uv run ruff check` → *All checks passed!*

☐ `uv run ruff format --check` → *already formatted*

☐ `uv run mypy` → *Success: no issues found*

☐ `uv run pytest` → tất cả *passed*

☐ `uv run pre-commit run --all-files` → tất cả *Passed*

☐ Commit lỗi bị pre-commit chặn (mục 10.3)

☐ Repo đã lên GitHub, có file `.github/workflows/ci.yml`

☐ Tab Actions có lần chạy ✅ trên `main`

☐ PR cố ý gây lỗi hiện ❌, sửa xong chuyển ✅ (mục 13.2)

☐ README có badge CI

15.2. Bài tập mở rộng

1. **Thêm quy tắc:** bật thêm nhóm `"D"` (docstring) trong Ruff, xem có bao nhiêu lỗi mới và sửa hết.
2. **Đo độ phủ test:** `uv add --dev pytest-cov`, đổi bước test trong CI thành `uv run pytest --cov=demo_ci --cov-report=term-missing`.
3. **Chạy trên nhiều hệ điều hành:** thêm `os: [ubuntu-latest, windows-latest, macos-latest]` vào `matrix` và đổi `runs-on: ${{ matrix.os }}`.
4. **So sánh type checker:** chạy `uvx ty check` và `uvx pyright` trên cùng dự án, ghi nhận khác biệt so với mypy.
5. **Tự động cập nhật phụ thuộc:** tìm hiểu Dependabot hoặc Renovate hỗ trợ `uv.lock`.

Phụ lục A – Toàn bộ file cấu hình

`pyproject.toml` hoàn chỉnh

```toml
[project]
name = "demo-ci"
version = "0.1.0"
description = "Demo linter, type checker & CI với uv"
readme = "README.md"
requires-python = ">=3.13"
dependencies = []

[build-system]
requires = ["uv_build>=0.x,<0.y"]   # giữ nguyên dòng uv init đã tạo
build-backend = "uv_build"

[dependency-groups]
dev = [
    "mypy>=2.4.0",
    "pre-commit>=4.7.0",
    "pytest>=9.1.1",
    "ruff>=0.17.0",
]

[tool.ruff]
line-length = 88
target-version = "py313"
src = ["src", "tests"]

[tool.ruff.lint]
select = ["E", "W", "F", "I", "B", "UP", "SIM", "N"]

[tool.ruff.format]
quote-style = "double"

[tool.mypy]
python_version = "3.13"
strict = true
files = ["src", "tests"]

[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = "-q"
```

`.pre-commit-config.yaml` và `.github/workflows/ci.yml`

Xem nguyên văn ở **mục 10.1** và **mục 12**. Cả hai file đã được kiểm tra bằng `pre-commit run --all-files` và `actionlint` không có lỗi.

Cấu trúc dự án cuối cùng

```
demo-ci/
├── .github/
│   └── workflows/
│       └── ci.yml
├── .gitignore
├── .pre-commit-config.yaml
├── .python-version
├── README.md
├── pyproject.toml
├── uv.lock
├── src/
│   └── demo_ci/
│       ├── __init__.py
│       ├── calculator.py
│       └── py.typed
└── tests/
    └── test_calculator.py
```

Phụ lục B – Bảng lệnh nhanh

Mục đích	Lệnh
Tạo dự án thư viện	`uv init --lib --python 3.13 <tên>`
Thêm thư viện chạy chính	`uv add <gói>`
Thêm công cụ dev	`uv add --dev <gói>`
Gỡ thư viện	`uv remove <gói>`
Đồng bộ môi trường theo lock	`uv sync`
Cập nhật file lock	`uv lock` / nâng cấp hết: `uv lock --upgrade`
Chạy lệnh trong môi trường	`uv run <lệnh>`
Chạy công cụ một lần, không cài vào dự án	`uvx <công-cụ>`
Lint / tự sửa	`uv run ruff check` / `uv run ruff check --fix`
Format / kiểm tra format	`uv run ruff format` / `uv run ruff format --check`
Tra cứu một quy tắc Ruff	`uv run ruff rule <MÃ>`
Kiểm tra kiểu	`uv run mypy`
Chạy test	`uv run pytest`
Cài hook pre-commit	`uv run pre-commit install`
Chạy hook trên mọi file	`uv run pre-commit run --all-files`

Mục đích	Lệnh
Chạy toàn bộ kiểm tra như CI	`uv run ruff check && uv run ruff format --check && uv run mypy && uv run pytest`

Tài liệu tham khảo

- uv – Cài đặt: docs.astral.sh/uv/getting-started/installation/
- uv – Dùng với GitHub Actions: docs.astral.sh/uv/guides/integration/github/
- uv – Dùng với pre-commit: docs.astral.sh/uv/guides/integration/pre-commit/
- Ruff – Tài liệu và danh sách quy tắc: docs.astral.sh/ruff/rules/
- mypy – Tài liệu: mypy.readthedocs.io
- setup-uv action: github.com/astral-sh/setup-uv
- GitHub Actions – Tài liệu: docs.github.com/actions
- pre-commit: pre-commit.com
- ty (type checker của Astral): docs.astral.sh/ty/